Evaluación de Unidad

1. Datos Generales

Asignatura	Algebra Lineal
Ciclo	**1**er Ciclo Computación
Unidad	**1**
Resultado de aprendizaje de la unidad	Aplica y resuelve analíticamente utilizando diversos métodos sistemas de ecuaciones lineales, matrices y determinantes, aplicaciones lineales según diversos criterios y utilizando diversas técnicas para la resolución de problemas con ayuda de software matemático, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Nombre del Docente	**Edwin Renè Guaman Quinche**
Nombre del Estudiante	**Anthony Eduardo Reyes Bravo**
Fecha	**04 al 10 de noviembre de 2026**

2. Objetivo

- Demostrar la capacidad de identificar, modelar matemáticamente y resolver un problema práctico utilizando el álgebra lineal aplicando sistemas de ecuaciones lineales o matrices.

3. Contexto

En este proyecto se analiza y aplica la teoría del álgebra lineal en la computación gráfica y el desarrollo de videojuegos mediante la creación de la aplicación interactiva **Matrix Snake** (*La Serpiente Matricial*) en la plataforma Google Colab. Elegimos este tema porque permite convertir un entorno virtual dinámico en una estructura puramente matemática: modelamos el mapa como una matriz discreta $M \in \mathbb{R}^{10\times10}$, representamos las coordenadas de la serpiente y la fruta como vectores en $\mathbb{R}^2$, calculamos las rotaciones de dirección mediante producto de matrices y renderizamos la pantalla usando tensores de color RGB ($10 \times 10 \times 3$).

Este trabajo tiene relevancia en el ámbito de la Ingeniería en Computación, ya que ilustra de qué manera los motores de gráficos y las simulaciones físicas sustentan la visualización y la interacción instantánea a partir del cálculo de matrices. A través de esta tarea, adquiriremos habilidades para comprobar analíticamente los cálculos vectoriales hechos a mano en contraste con su ejecución en Python utilizando NumPy, lo que nos permitirá entender cómo se aplican las transformaciones lineales, la adición de vectores y el manejo de datos en múltiples dimensiones en el proceso de creación de software auténtico.

4. Desarrollo de las operaciones

En este proyecto se realizarán las siguientes operaciones:

- **Multiplicación de una matriz por un vector:** Producto de la matriz de rotación 2×2 por el vector columna de dirección 2×1 para calcular el cambio de orientación.
- **Suma de vectores:** Adición componente a componente del vector de posición actual y el nuevo vector de dirección para determinar la traslación.
- **Mapeo de estado matricial:** Representación de los elementos en matrices discretas $\mathbf{M_0}, \mathbf{M_1} \in \mathbb{R}^{10\times10}$.

El sistema de movimiento y rotación se compone por los siguientes elementos vectoriales:

- **Vector desplazamiento:**

$$\mathbf{D} = \begin{bmatrix} V_f \\ V_c \end{bmatrix}$$

- Vf (componente fila)
- Vc (componente columna)

- **Vectores Canónicos de Desplazamiento:**

- Derecha: D derecha = $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$
- Arriba: d arriba = $\begin{bmatrix} -1 \\ 0 \end{bmatrix}$
- Izquierda: D izquierda = $\begin{bmatrix} 0 \\ -1 \end{bmatrix}$
- Abajo: D abajo = $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$

- **Traslación (Suma de Matrices):**

La posición final de la cabeza (Pnueva) resulta de la suma entre su posición actual (Pnueva) y el vector desplazamiento (D)

P nueva = P actual + D

$$\begin{bmatrix} F_{\text{nueva}} \\ C_{\text{nueva}} \end{bmatrix} = \begin{bmatrix} F_{\text{actual}} \\ C_{\text{actual}} \end{bmatrix} + \begin{bmatrix} V_f \\ V_c \end{bmatrix} = \begin{bmatrix} F_{\text{actual}} + V_f \\ C_{\text{actual}} + V_c \end{bmatrix}$$

- **Rotación (Producto Matriz por Vector):**

$$\mathbf{D}_{\text{nueva}} = \mathbf{R}_\theta \times \mathbf{D}_{\text{actual}}$$

$$\begin{bmatrix} V_f' \\ V_c' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} V_f \\ V_c \end{bmatrix} = \begin{bmatrix} (V_f \cdot \cos\theta) - (V_c \cdot \sin\theta) \\ (V_f \cdot \sin\theta) + (V_c \cdot \cos\theta) \end{bmatrix}$$

Giro a la izquierda ($90°$ **antihorario):** $\theta = 90° \Longrightarrow \mathbf{R}_{\text{izq}} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

Giro a la derecha ($90°$ **horario):** $\theta = -90° \Longrightarrow \mathbf{R}_{\text{der}} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$

4.1. Planteamiento del problema

En el desarrollo de entornos gráficos 2D, el reto principal es coordinar el movimiento, la orientación de los objetos y la renderización en pantalla mediante modelos algebraicos exactos sin depender de estructuras lógicas complejas. En **Matrix Snake**, el espacio de juego es un tablero bidimensional de 10×10 casillas.

4.2. Procedimiento paso a paso

- **Posición inicial del cuerpo de la serpiente:**
  - **Cabeza (P_0):** $\begin{pmatrix}5\\5\end{pmatrix}$ (Fila 5, Columna
    Se representa con valor 1
  - **Cuerpo:** $\begin{pmatrix}5\\4\end{pmatrix}$ (Fila 5, Columna 4)
    Se representa con valor 2
  - **Cola:** $\begin{pmatrix}5\\3\end{pmatrix}$ (Fila 5, Columna 3)
    Se representa con valor 2

$$\begin{bmatrix}0&0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\\0&0&\mathbf{9}&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\\0&0&0&\mathbf{2}&\mathbf{2}&\mathbf{1}&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0&0\end{bmatrix}$$

Fig 1: Cabeza en posición (5,5), Cuerpo en posición (5,4) y cola en posición (5,3)

- **Vector de dirección inicial (v_0):**

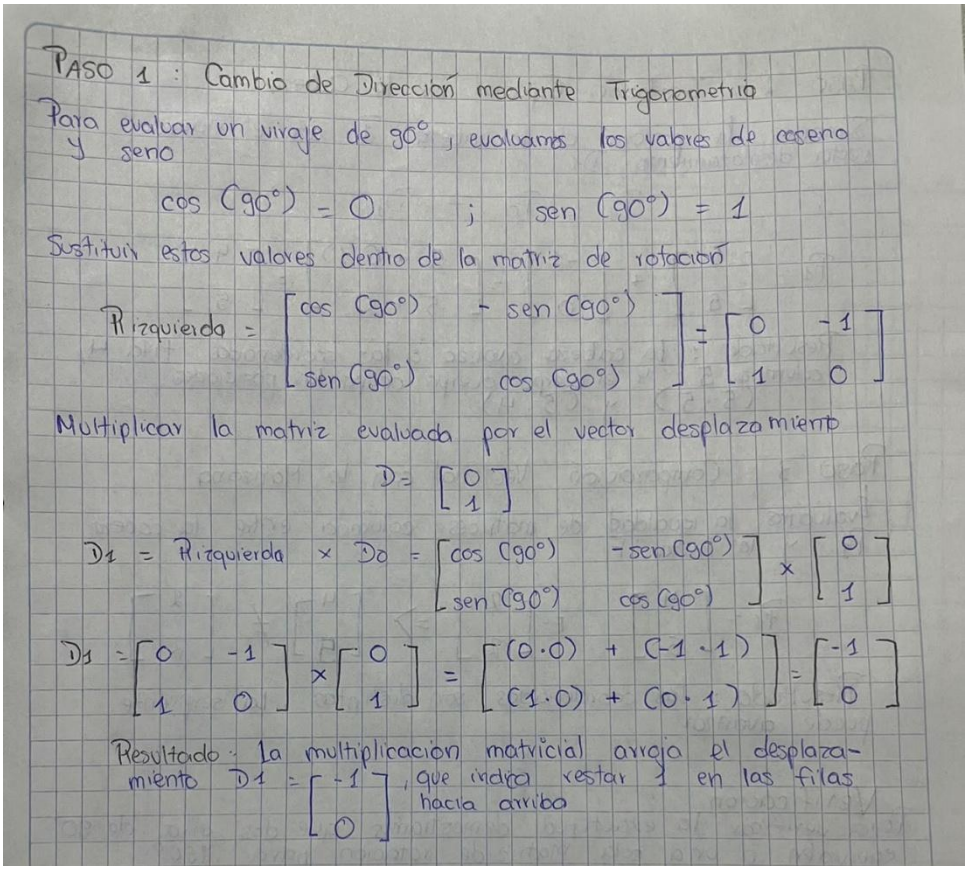
$$\mathbf{v}_0 = \begin{pmatrix}0\\1\end{pmatrix} \quad \text{(Hacia la derecha)}$$

- **Vector de posición de la fruta (F):**

$$\mathbf{F} = \begin{pmatrix}2\\2\end{pmatrix} \quad \text{(Fila 2, Columna 2)}$$

Se representa con el valor 9

- **PASO 1**



PASO 1 : Cambio de Dirección mediante Trigonometría

Para evaluar un viraje de 90°, evaluamos los valores de coseno y seno

$$\cos(90°) = 0 \quad ; \quad \text{sen}(90°) = 1$$

Sustituir estos valores dentro de la matriz de rotación

$$R_{izquierda} = \begin{bmatrix} \cos(90°) & -\text{sen}(90°) \\ \text{sen}(90°) & \cos(90°) \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Multiplicar la matriz evaluada por el vector desplazamiento

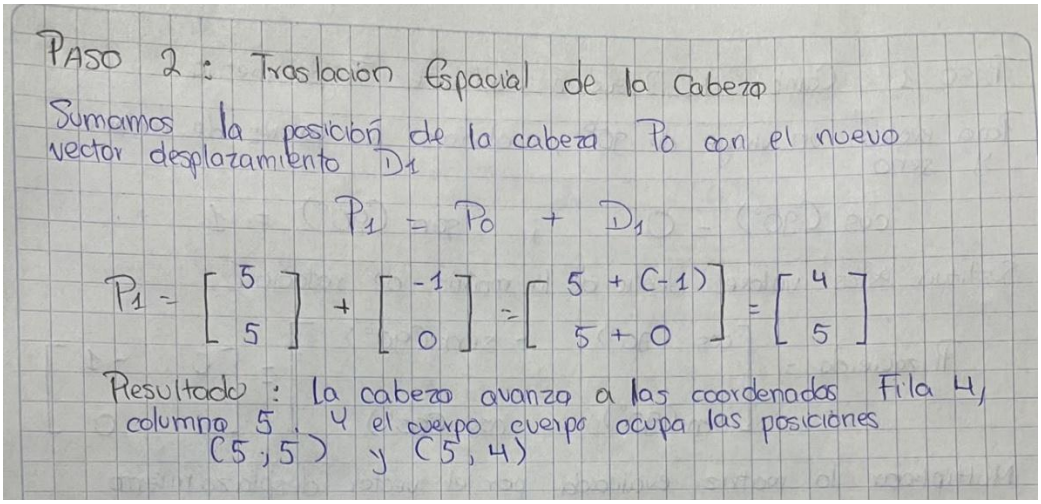
$$D = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$D_1 = R_{izquierda} \times D_0 = \begin{bmatrix} \cos(90°) & -\text{sen}(90°) \\ \text{sen}(90°) & \cos(90°) \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$D_1 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} (0 \cdot 0) + (-1 \cdot 1) \\ (1 \cdot 0) + (0 \cdot 1) \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

Resultado: la multiplicación matricial arroja el desplazamiento $D1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$, que indica restar 1 en las filas hacia arriba

- **PASO 2**



PASO 2 : Traslación Espacial de la Cabeza

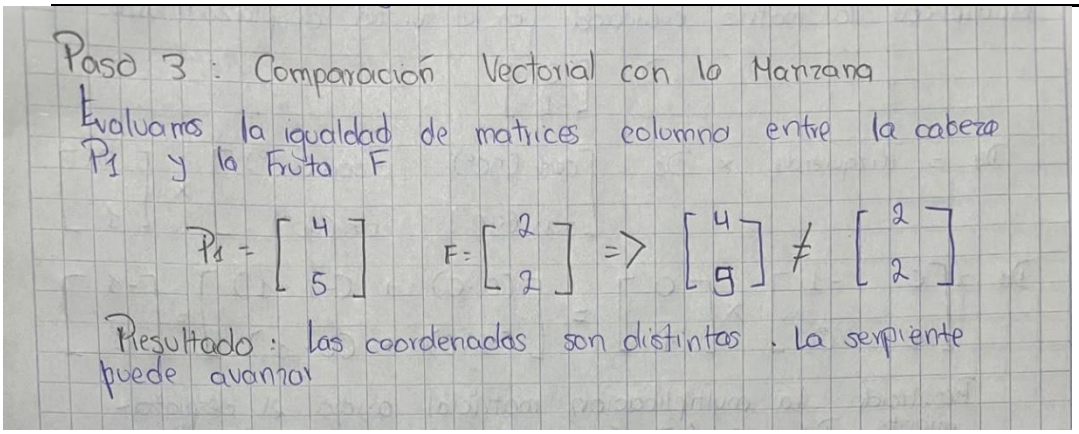
Sumamos la posición de la cabeza P_0 con el nuevo vector desplazamiento D_1

$$P_1 = P_0 + D_1$$

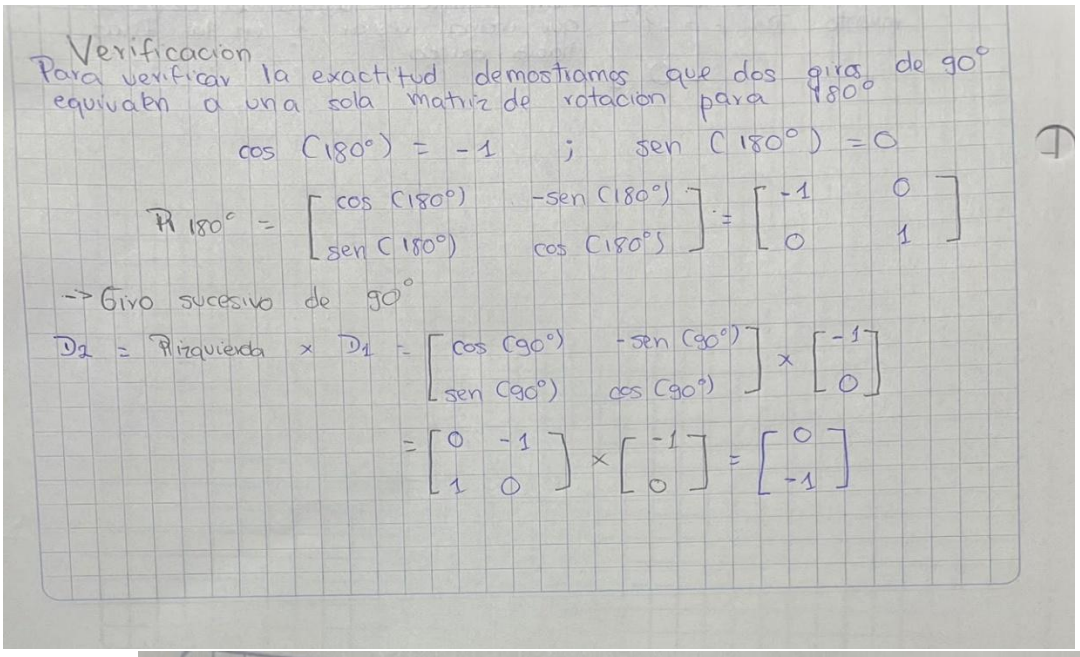
$$P_1 = \begin{bmatrix} 5 \\ 5 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 + (-1) \\ 5 + 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

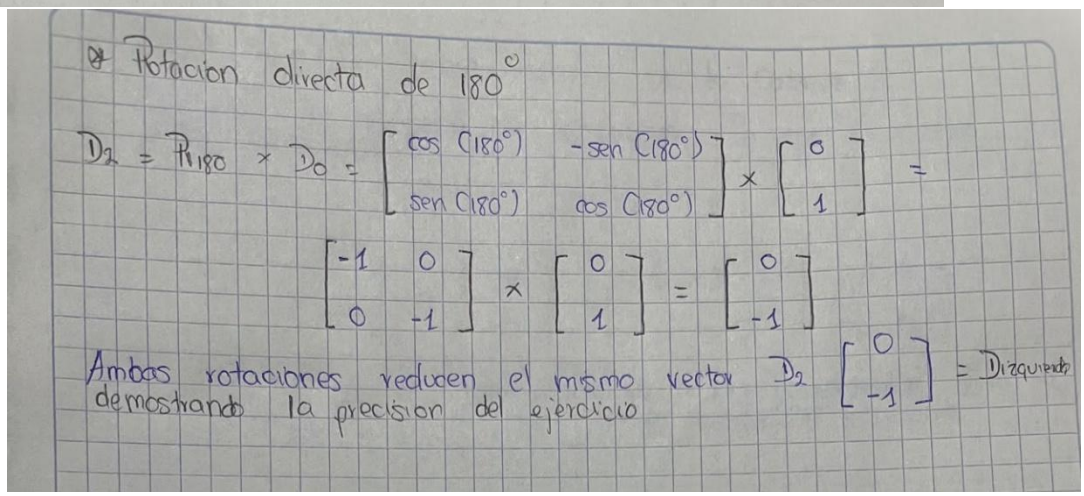
Resultado : la cabeza avanza a las coordenadas Fila 4, columna 5 , 4 el cuerpo cuerpo ocupa las posiciones (5 ; 5) y (5, 4)

- **PASO 3**



4.3 Verificación de resultados





5. Procedimiento / Metodología

Para la resolución del programa se utilizó la siguiente metodología:

- Primeramente, se seleccionó el tema el cual fue del juego de la serpiente en un entorno de 2D.
- Para la resolución del juego se utilizo una matriz de 10 * 10. Para el giro de la dirección de modelo el producto de matrices 2 * 2 por 2 * 1 con operadores trigonométricos
- Mediante la ayuda de la IA, se estructuro el código modulado importando **NumPy** para el calculo de matrices, Matplotlib para el renderizado del mapa y ipywidgets para habilitar los controles para poder jugarlo

5.1 Descripción de la aplicación

- **¿En qué consiste'**

El problema consiste en un juego interactivo en Google colab denominado Matrix Snake que muestra el tablero del juego, muestra la figura de la serpiente y la fruta representados en ese programa.

- **¿Qué problema resuelve?**

El programa en si controla y calcula hacia donde mira la cabeza de la serpiente y como se mueve alrededor de todo el mapa para alcanzar su objetivo que es la manzana. También resuelve el cálculo y el viraje del personaje empleando solo operaciones algebraicas.
El sujeto automatiza las colisiones contra los bordes del mapa para evitar que salga de la matriz 10*10.

- **¿A quién va dirigida?**

Esta orientada a estudiantes, docentes y programadores interesados en comprender como los principios de Algebra lineal se implementan en la creación de juegos o programas.

- **Objetivo principal**

Demostrar que las operaciones entre vectores son instrumentos para poder crear el movimiento de cualquier objeto.

- **Relación con Algebra lineal:**

Se relaciona ya que permite el avance del personaje se ejecuta por medio de la suma de vectores entre posición y vector dirección.
Los giros de 90* se ejecutan mediante el producto de matrices el cual son temas principales de la algebra lineal lo que permite que la serpiente gire a la izquierda o derecha

5.2 Descripción de la aplicación

Categoría	Herramienta / Tecnología	Versión / Detalle
Lenguaje de programación	[Ej. Python, Java, JavaScript, etc.]	[Versión]
Entorno de desarrollo (IDE)	[Ej. VS Code, PyCharm, Google Colab, etc.]	
Librerías / Frameworks	[Ej. NumPy, Panda, React, etc.]	[Versión]

Plataforma de ejecución	[Ej. Escritorio, Web, Móvil]	

5.3 Funcionalidades implementadas

Lista las funcionalidades principales que tiene la aplicación. Describe brevemente qué hace cada una y qué concepto de Álgebra Lineal aplica.

- **Funcionalidad 1: Reorientación del sujeto:**

Permite cambiar el rumbo del personaje 90* a la izquierda o a la derecha al presionar los botones de giro, calculando el producto entre la matriz de rotación y el vector de dirección.
Aplica al algebra lineal ya que utiliza producto de matrices 2*2 por 2*1.

- **Funcionalidad 2: Traslación Espacial:**

Permite calcular el avance cuadro a cuadro sobre el tablero-matriz de 10 * 10. Realizando suma de matrices entre el vector posición actual y el vector desplazamiento.
Se relaciona con la algebra ya que aplica la suma de vectores-matrices.

- **Funcionalidad 3: Renderizado de Gráficos con tensores RGB:**

Construye la representación de como se ve el mapa en la ejecución del programa, asignando a cada casilla de la matriz 10*10 un vector tridimensional, para diferenciar la cabeza, cuerpo, la manzana y el fondo.

5.4 Capturas de pantalla

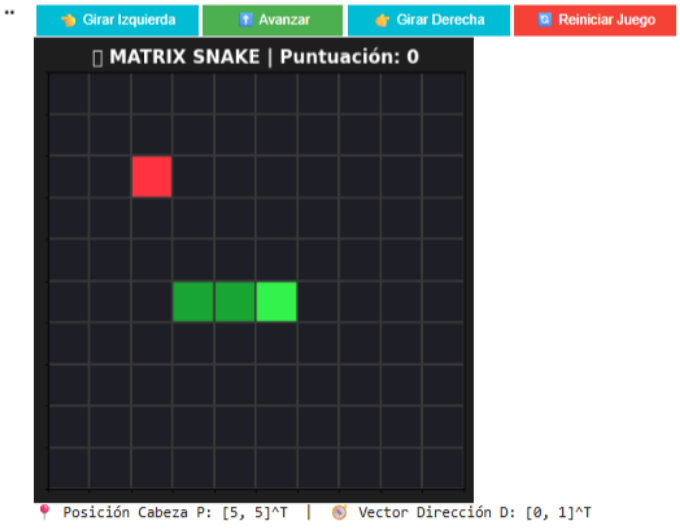


Figura 1: Interfaz interactiva de Matrix snake. Muestra la serpiente en la posición inicial Po = 85,5) y el vector dirección

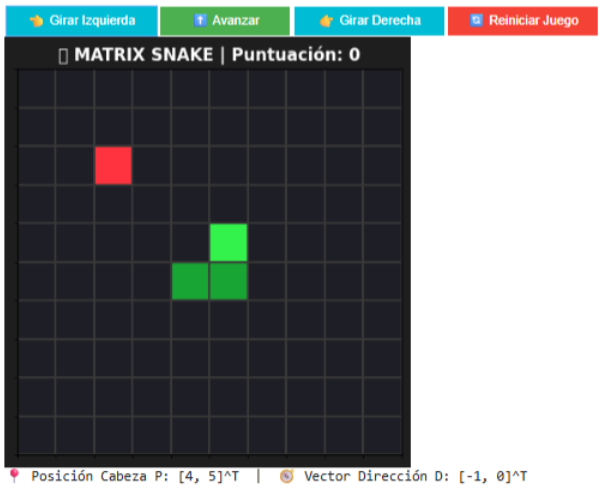


Figura 2:Se muestra el estado del juego al presionar Girar a la izquierda demostrando la multiplicación por la matriz de rotación R90 y se muestra la traslacion espacial por suma de vectores a las coordenadas (4,5)

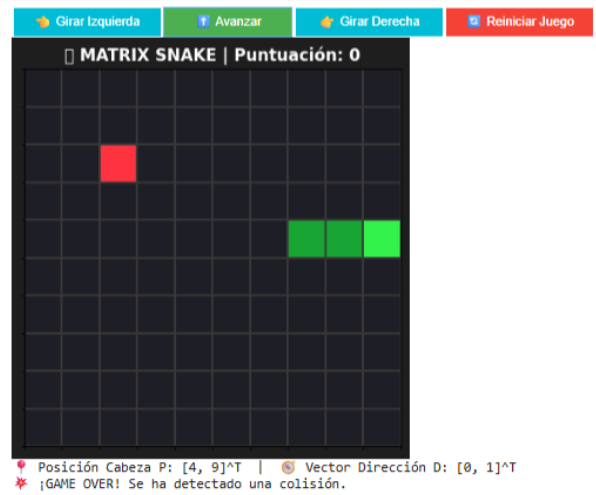


Figura 3; Se demuestra los limites dentro del tablero 10*10

5.5 Código fuente (fragmentos clave)

```python
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output

# ---------------------------------------------------------
# 1. DATOS DE ENTRADA Y ESTADO INICIAL DEL EJERCICIO (t=0)
# ---------------------------------------------------------
TAMANO = 10

# Posiciones iniciales exactas del problema analítico
serpiente = [[5, 5], [5, 4], [5, 3]]  # Cabeza P0 = [5, 5]^T, Cuerpo = [5, 4]^T y [5, 3]^T
direccion = np.array([0, 1])          # Vector dirección D0 = [0, 1]^T (Avanzar hacia la derecha)
fruta = [2, 2]                        # Posición fija F = [2, 2]^T
puntuacion = 0
juego_terminado = False
```

Figura 1: Tamaño del tablero, posición de la cabeza y el vector movimiento inicial

```python
# Giro a la izquierda: theta = 90 deg -> cos(90)=0, sin(90)=1
# R_izq = [[cos(90), -sin(90)], [sin(90), cos(90)]] = [[0, -1], [1, 0]]
ROT_IZQUIERDA = np.array([[0, -1], [1, 0]])

# Giro a la derecha: theta = -90 deg -> cos(-90)=0, sin(-90)=-1
# R_der = [[cos(-90), -sin(-90)], [sin(-90), cos(-90)]] = [[0, 1], [-1, 0]]
ROT_DERECHA = np.array([[0, 1], [-1, 0]])
```

Figura 2: Matrices de transformación de rotación

```python
def girar_izquierda(b):
    global direccion
    # Producto matricial: np.dot(R_izq, D_actual)
    direccion = np.dot(ROT_IZQUIERDA, direccion)
    dar_paso()
```

Figura 3:se define la matriz rotacion a la izquierda

```python
def dar_paso():
    """
    Paso 2 del ejercicio: Traslación Espacial por Suma de Vectores
    P_nueva = P_actual + D
    """
    global serpiente, puntuacion, juego_terminado
    if juego_terminado: return

    cabeza_actual = np.array(serpiente[0])
    # Suma matricial de vectores columna
    nueva_cabeza = (cabeza_actual + direccion).tolist()

    # Validación de límites de la matriz y colisión con el cuerpo
    if not (0 <= nueva_cabeza[0] < TAMANO and 0 <= nueva_cabeza[1] < TAMANO) or (nueva_cabeza in serpiente):
        juego_terminado = True
        renderizar()
        return

    serpiente.insert(0, nueva_cabeza)

    # Paso 3 del ejercicio: Condición de Coincidencia (Igualdad Vectorial P == F)
    if nueva_cabeza == fruta:
        puntuacion += 10
        # Nueva fruta en el subespacio discreto
        while True:
            nueva_f = [np.random.randint(0, TAMANO), np.random.randint(0, TAMANO)]
            if nueva_f not in serpiente:
                fruta[0], fruta[1] = nueva_f
                break
    else:
        serpiente.pop()
```

Figura 4: Se calcula la nueva coordenada de la cabeza mediante la sema vectorial mencionada en el proceso anterior

Repositorio / Enlace al proyecto:

Enlace Google Colab: https://colab.research.google.com/drive/1-8dX6u4OXWqO2MbjZatff9MCMSAPBZJO#scrollTo=L4RcO5j6f6xo

Enlace github: https://github.com/Anthonyr18/Algebra-lineal.git

6. Resultados

Reflexiona sobre los resultados obtenidos tanto en la parte manual como en la aplicación computacional. Responde las siguientes preguntas orientadoras:

• ¿Los resultados del cálculo manual coinciden con los de la aplicación? ¿Qué diferencias existen y a qué se deben?

Sí, existe coincidencia absoluta. El desarrollo a mano evaluó $\cos(90°) = 0$ y $\sin(90°) = 1$ para formar la matriz R_{izq} y multiplicar por la dirección $\begin{pmatrix}0\\1\end{pmatrix}$, obteniendo el vector resultante $\begin{pmatrix}-1\\0\end{pmatrix}$ que sitúa la cabeza en la casilla $(4,5)$. En la aplicación, la instrucción np.dot(ROT_IZQUIERDA, direccion) ejecuta exactamente el mismo producto matricial con idénticos valores en pantalla.

• ¿Cuáles fueron las mayores dificultades que encontraste al resolver el problema manualmente? ¿Y al programarlo?

En si la resolución del ejercicio manual no hubo problema realizarlo la cosa cambio a la horde juegos a de programación., pero como se la realizo con ayuda de ia cambio un poco la dificultad

• ¿En qué situaciones reales de la computación o ingeniería se aplica el tema que estudiaste?

Se aplica directamente al desarrollo de mototres graficos en 2D y 3D, en la creación de programas y juegos

• ¿Qué aprendiste al construir la aplicación que no hubieras aprendido solo con teoría?

Entendimos al fin cómo funciona una pantalla digital. Cada una opera como un tensor matricial de tres dimensiones compuesto por píxeles.

Para que cualquier programa tenga movimiento y una física creíble, las transformaciones lineales son la pieza fundamental. Son la herramienta de base.

7. Conclusiones

Presenta al menos 1 conclusión concretas derivadas del trabajo realizado. Las conclusiones deben ser específicas, derivadas de la experiencia del proyecto (no afirmaciones genéricas).

- Primera conclusión: Para manejar el rumbo de objetos en planos de dos dimensiones, lo más práctico es usar matrices de rotación trigonométrica con sus senos y cosenos. Al multiplicar estas matrices, obtienes un procedimiento riguroso para controlar cualquier desplazamiento. Es el sistema que mejor funciona.
- Segunda conclusión: Con el proyecto **Matrix Snake** pudimos comprobar de forma súper práctica que el álgebra lineal es la base con la que funcionan los gráficos y la física de los videojuegos.

  Al terminar el proyecto Matrix Snake, quedó claro que el Álgebra Lineal no es solo un montón de fórmulas en una pizarra. Es el motor real que mueve las figuras, gestiona los choques entre objetos y calcula cada movimiento en los juegos que vemos en pantalla.

- Este estudio confirma que no hacen falta pruebas prácticas para resolver los desplazamientos en el plano. Basta con sumar vectores columna y aplicar matrices de rotación de 2x2 para obtener resultados exactos. Es un procedimiento puramente matemático.

8. Referencias

[1] "Vectores y Matrices : Curso de programación lógica básica a avanzada | Cursa," *Cursa*. https://cursa.app/es/pagina/vectores-y-matrices

[2] "¿Qué es Python? | Oracle América Latina." https://www.oracle.com/latam/developer/what-is-python-for-developers/